Stepper Motor with Filter Wheel

Karaduman Burhan

Middle East Technical University, Thermally and Optically Stimulated Luminescence Laboratory Research Project

Contests

1. Abstract 1

2. Introduction 1

2.1 Problem Statement: Optical Filtering in TOSL Research 1

2.2 Project Objectives: Automated Design and Precision 1

3. Methodology 2

3.1 Hardware Design 2

3.1.1 Component Analysis and Motor Specifications 2

3.1.2 Power Supply and Driver Configuration (L293D) 2

3.1.3 Optocoupler Integration and Sensor Circuitry 2

3.1.4 Port Expansion via PCF8574 (I^2C Protocol) 3

3.1.5 Shield Design and Wiring Efficiency 3

3.1.6 Power Distribution and Signal Debouncing 4

3.1.7 Signal Stability: Addressing Floating Pins 4

3.1.8 Assembly Challenges and Soldering Techniques 4

3.2 Software Development 4

3.2.1 System Architecture and Library Integration 4

3.2.2 Homing Routine and Interrupt Service Routines (ISR) 5

3.2.3 Position Calculation and Path Optimization 5

3.2.4 Custom Class Override for PCF8574 Integration 6

3.2.5 Debugging and Logic Correction 7

3.2.6 Input Validation and Error Handling 7

3.2.7 Final Software Stability and Performance 7

4. Results and Discussion 8

4.1 System Performance and Positioning Accuracy 8

4.2 Comparison with Manual Methods 8

5. References 9

6. Appendices 10

Appendix A: Hardware Schematics and Pinouts

Figure 5: Cable Pin Connections

Figure 6: All Pins Output and Input Schematic

Figure 7- 8: Complete I/O Schematic

Appendix B: Project Photos 11

Figure 9 & 10: Filter Wheel Mechanism

Appendix C: Source Code 12

1. Abstract

This report details the design and implementation of an automated filter wheel system intended for use in Thermally and Optically Stimulated Luminescence (TOSL) laboratory research. To ensure precise optical filtering and reduce manual intervention during experiments, an Arduino Nano microcontroller was selected as the central control unit. The system utilizes a stepper motor to achieve accurate positioning for multiple optical filters. A key feature of the design is the integration of a PCF8574 port expander, which manages motor control and user inputs via the I²C protocol, significantly reducing wiring complexity and preserving microcontroller resources. Furthermore, an OPB931 slotted optocoupler was implemented to establish a reliable "Home" (zero) position through interrupt-based detection. The final prototype provides a stable, user-friendly interface with both serial command and physical button control, offering a consistent solution for automated wavelength selection .

2. Introduction

2.1 Optical filtering plays a critical role in precise laboratory experiments, particularly in Thermally and Optically Stimulated Luminescence (TOSL) research, where isolating specific light wavelengths is essential. Performing these filter changes manually can be time-consuming and prone to alignment errors, potentially compromising experimental data. To address these challenges, this project aims to develop a fully automated, electronically controlled filter wheel system.

2.2 The primary objective was to create a device that allows researchers to swhich between filters rapidly and accurately without physical handling. The system is built upon the Arduino platform, chosen for its flexibility and extensive library support. The design prioritizes mechanical precision and hardware efficiency; therefore, a stepper motor is used for exact rotational steps , and a modular circuit design is employed to minimize cable clutter. Additionally, real-time system feedback is provided to the user through an LCD screen. This report outlines the complete development process, from the selection of electrical components and circuit assembly to the implementation of software algorithms for reliable motor control.

3. Methodology

3.1 Hardware Design

3.1.1 In my project for electronic components, I found each component datasheet and note them. For example, my stepper motor is Portescap stepper motor and it has two phases yellow and black cables are for phase one, red and gray are for the phase two (chancing phase places are changing just direction) and its angle per step is 7.5 degrees so its total step for full turn circle is 48 but in my project I am using half step each time that mean I am using stepper motor like its total step like 96. However, there is a important information, each motor has special interval for working stable, for my stepper it is between 350–450 RPM, and if motor work outside of its own interval, motor can miss steps or can shake, for that I had to find it.

3.1.2 Also, I am using 12V for whole system and stepper motor current per phase 0.19A. My power supply is 12V, 1A DC, it is enough for the motor. However, we cannot connect power supply to motor directly. We have to use a motor driver for it because, as I explained in Introduction, stepper motors need to take voltage in order. Therefore, I used L293D motor driver which is enough for my stepper motor. Stepper motor was already connected to the filter wheel and on the wheel there is a optocoupler (OPB931) and it has two main part, lighter part which is give information and sensor part and it take information from lighter and give to the system.

3.1.3 Cathode have to connect to ground and Anode (red wire) which have to connected to the 5V but there is a important information, between Anode and 5V, there must be a 330 ohm resistor because these are lighter wires and these are give information to the sensor on optocoupler and when you give 5V directly its current will be very high and it can burn. White, green and blue cables are for sensor. Green cable is ground cable. White cable is for VCC and blue cable is for output which is send information from lighter, but there is really important information, there must be a parallel 10k resistor between VCC and OUTPUT. The resistor between VCC and OUTPUT is needed to convert the phototransistor's current change into a usable digital voltage signal. Moreover, blue cable have to connected to one of the Arduino nano pin, but I had to be careful because sensor work with Interrupt

mechanism and there are just two pins support this mechanism and they are Arduino nano digital pins D2 and D3 so it is important.

3.1.4 Also, I had to use port expander PCF8574. The PCF8574 is an I²C I/O expander. It allows a microcontroller to add 8 extra digital input/output pins using only two I²C lines: SDA (data) and SCL (clock). Each of the 8 pins (P0–P7) can be individually set as an input or output. Because it communicates through the I²C protocol, multiple PCF8574 chips can be connected on the same bus by changing their address pins (A0–A2), allowing easy port expansion without using many microcontroller pins. It's commonly used with Arduino, ESP32, and Raspberry Pi to control LCDs, buttons, or relays efficiently.

3.1.5 I used that because I had to use a lot of pin on Arduino nano, its pins are enough but I had to use a lot of different cable and it will make project complicate and inefficient, so I connected it on shield. If I didn't do that, I won't be able to create project effectively. Also, it just need 5V and ground supply, and 2 communication cable which is connected to the Arduino nano analog pins A4 (SDA) and A5 (SCL). Thus, I needed a lot of 5V and ground supply and if I wanted to take these supply I had to use a lot of cable, so I made Common supply on the PCB shield for them.

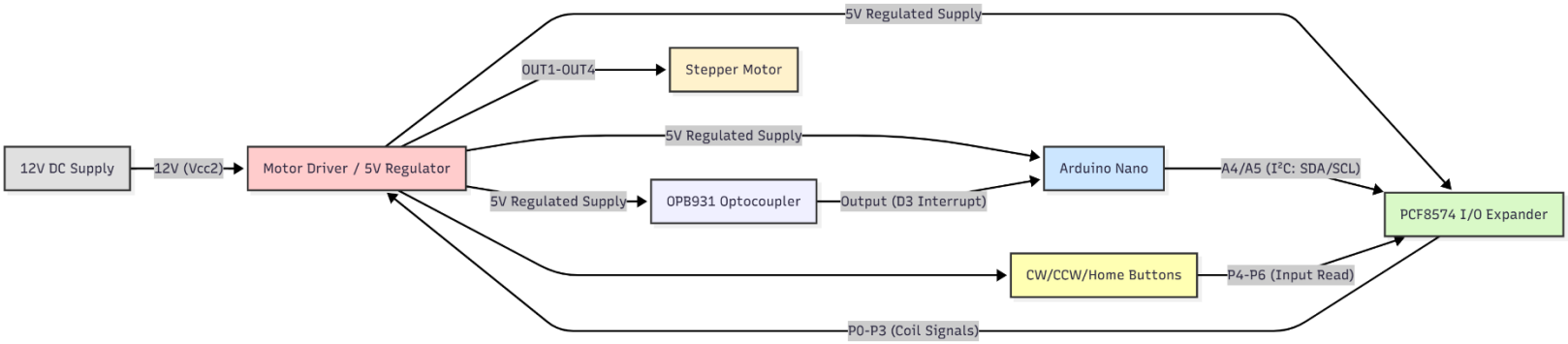

Figure 1: Block Diagram-This diagram illustrates the general architecture of the system, including the Arduino Nano, stepper motor driver, OPB931 sensor, PCF8574 I/O expander, and filter wheel. It provides a high-level overview of how power, control signals, and communication lines interact within the design.

3.1.6 I could do that because I was using 12V for my motor driver and on my motor driver there is 5V supply and ground. That driver can feed motors and at the same time, it can produce 5V supply. Therefore, I could just use 2 cable for power and made it command for all thing. Moreover, there are some buttons and I had to connected to them because one of them have to turn stepper motor clockwise when it pressed and other is counterclockwise, but buttons have problems and it is multiple press even when pressed one and its solution is Debounce. Debouncing is the process of filtering out extra signals, so the system detects only one stable input change. I solved that problem with software and basically I did I waited system 50 milliseconds for that.

3.1.7 Also there is one big problem too and it is Floating pin. A floating pin is a microcontroller input pin that is not connected to either a HIGH (VCC) or LOW (GND) voltage. When a pin is floating, its voltage can randomly change due to electrical noise. For that, pull up resistors can be used (with hardware or software).

3.1.8 After finishing prototype, I face a problem: I had to use cables shortly and components' places are most important one because I had to be careful about not blocking filter when it was working. I connected components on wrong places so I had to remove them. Also, when I was doing solder I had to be careful because I could burn components and cables. When cables soldered, we have to use heat shrink tubing because when cables move plastic parts will be very weak and in the future thy can break so we have to make them stronger. Also, we should twist them around each other because they will be stronger together. *Figure 1, Figure 5, Figure 6, Figure 7, Figure 8, and Figure 9–10* support the hardware design and show the wiring layout, block diagrams, and full schematic of the system.

3.2 SOFTWARE

3.2.1 In this project, I used Arduino based software system and it was developed to control a motorized filter wheel using a combination of I²C communication, stepper motor control, and serial input processing. My program integrates several libraries: Wire for I²C communication, PCF8574 to extend digital inputs and outputs for button and motor control, AccelStepper to manage smooth and precise stepper motor movement, SerialCommand to

process serial text commands, and LiquidCrystal_I2C to display system feedback such as current filter position or home status.

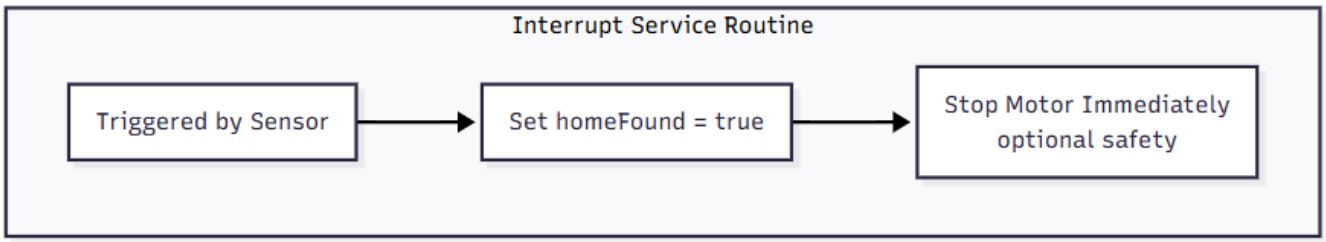

Figure 2: Interrupt Service Routine-This figure illustrates how the OPB931 sensor triggers the home position detection through a rising-edge interrupt, ensuring millisecond-level accuracy

3.2.2 Also, I used Interrupt service routine for finding and correcting home position (0 position) like shown in *Figure 2*. Therefore, I wrote a function for home position and it works according to Interrupt Service routine and optocoupler component. In Figure 3 it shown how it works.

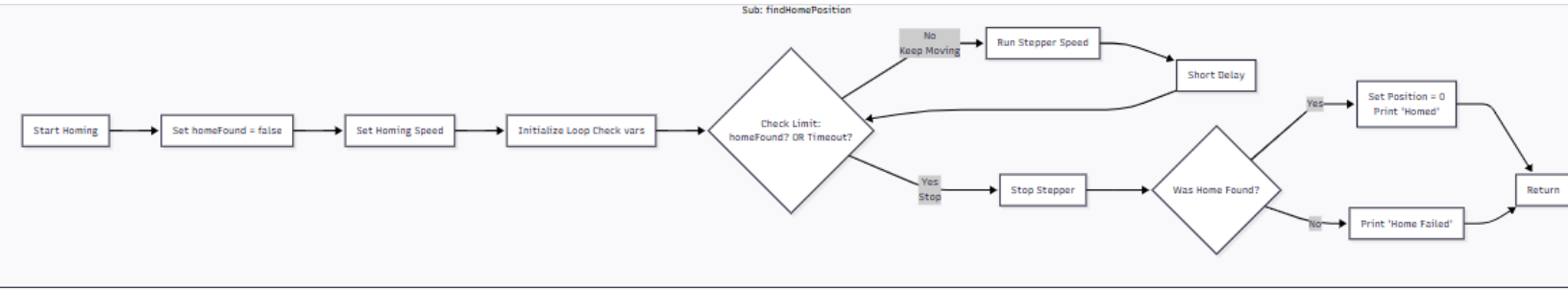

Figure 3: Home Position Function- This flowchart details the main software operations, including initialization, interrupt handling, filter position calculations, motor movement routines, LCD updates, and button/serial command processing. It corresponds to the software architecture.

3.2.3 My stepper motor positions for each optical filter (F1–F4) were predefined as specific step values, and the motor movement between these points and handling by a custom moveToFilter() function that calculates the shortest path using the current step position modulo the full wheel rotation according to button which one pressed.

3.2.4 I had to integrate the PCF8574 expander with the AccelStepper library, a custom subclass named PCFStepper was created. Inside this class, the setOutputPins(uint8_t mask) function was overridden and declared as protected, allowing direct control of the motor driver pins through the PCF8574 instead of the Arduino's digital pins. This function decodes the 4-bit mask generated by the AccelStepper library and writes each bit to the corresponding PCF8574 output using pcf.digitalWrite(0–3). By doing this, the standard AccelStepper step

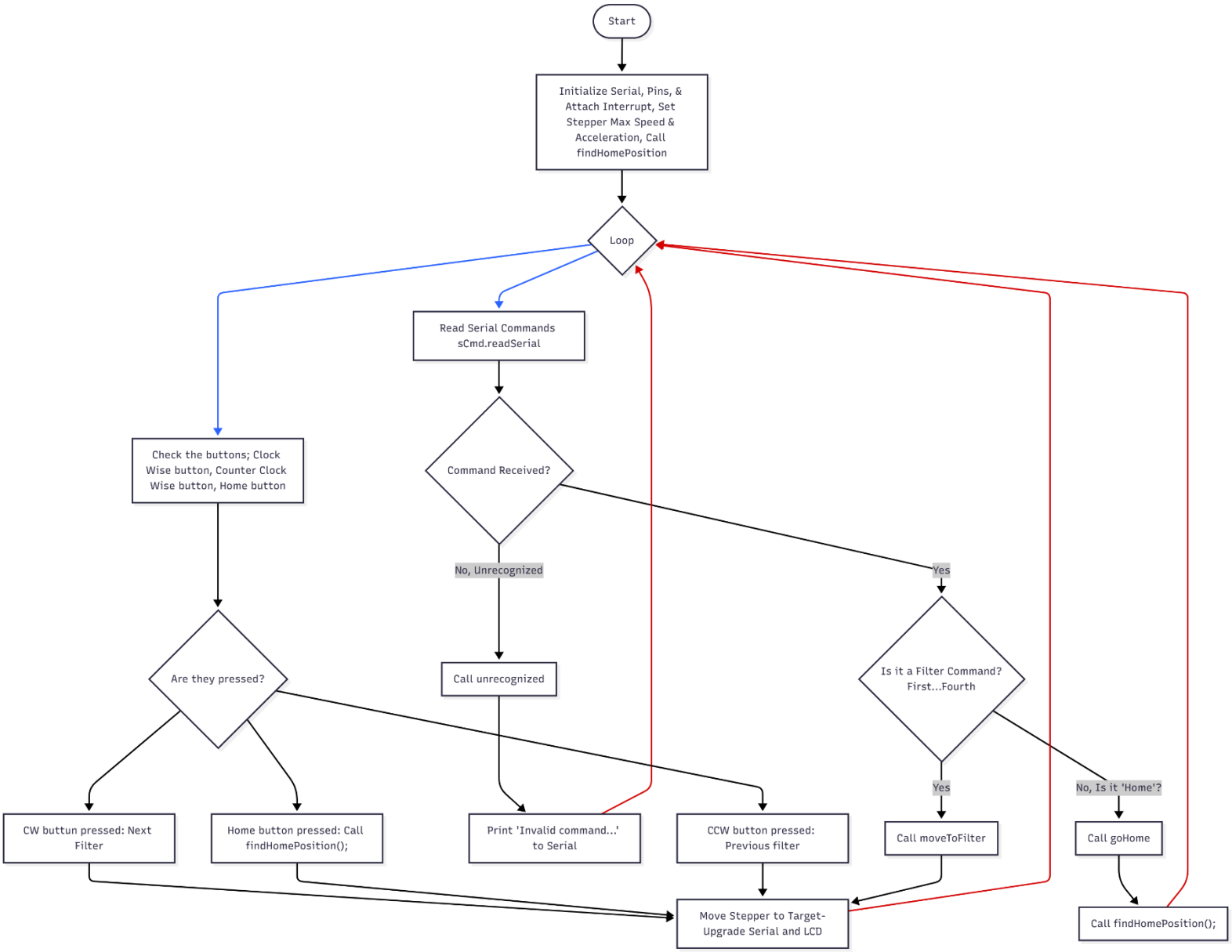

Figure 4-System Operation Diagram- summarizes the interaction between hardware and software during runtime. It shows how the system detects home position, processes inputs, calculates movements, and drives the stepper motor to the correct filter location.

sequences are sent through the I²C bus, effectively enabling full stepper control using the port expander without losing timing precision. I did that because AccelStepper library just work with digital signal by Arduino, so I had to override it.

3.2.5 When I was testing software, I faced several programming issues. For example, when buttons pressed program was struggling calculate short distance between filters and I found it happened because of just one condition. Also, the LCD initially failed to display properly because it was initialized before Wire.begin(), which was corrected by reordering the setup sequence. My pressCount variable that tracks filter changes occasionally overflowed or desynchronized with the LCD display, so boundary conditions and display updates were added after each completed movement.

3.2.6 Serial commands like "SPEED" and "ACCEL" caused runtime errors when arguments were missing, which were resolved by adding input validation. The home detection function homeFound was also found to stay true after homing, preventing subsequent home searches; resetting it at the end of findHomePosition() fixed the issue. Lastly, button input instability caused by signal bouncing was mitigated with a short delay (dt) in the main loop.

3.2.7 After these improvements, the software became stable and responsive, the stepper motor accurately moved to each predefined position via the PCF8574, LCD updates reflected real-time motor states, and both serial and manual controls worked reliably and consistently. *Figure 4* shows the full software flowchart with command parsing and interrupt-based homing logic.

4. RESULT and DISCUSSION

After completing both the hardware assembly and software debugging, the system worked as expected. The stepper motor accurately moved the filter wheel to each predefined position (F1 –F4) and successfully returned to the home position detected by the OPB931 sensor. The LCD displayed real-time information such as “On Filter X” or “On Home,” confirming correct synchronization between the motor and display. The integration between the PCF8574 expander and the AccelStepper library allowed smooth communication over the I^2C bus without timing errors, proving that the overridden setOutputPins() function worked effectively. During operation, the system showed stable performance with no signal noise or step loss after implementing debounce delays and input validation. Compared to previous manual control systems, this design achieved faster and more reliable positioning with reduced wiring complexity and higher precision. The final prototype demonstrated efficient, repeatable, and accurate control of the filter wheel through both serial commands and physical buttons.

REFERENCES

[1] Texas Instruments, "L293D Quadruple Half-H Drivers," Datasheet, 1999. [Online]. Available: https://www.ti.com/lit/ds/symlink/l293d.pdf

[2] Philips Semiconductors, "PCF8574: Remote 8-bit I/O expander for I2C-bus with interrupt," Datasheet. [Online]. Available: https://www.alldatasheet.com/datasheet-pdf/view/18212/PHILIPS/PCF8574.html. [Accessed: Oct. 8, 2025].

[3] Portescap, "Can Stack Stepper Motors 35L048B: Technical Specifications," Datasheet, 2024. [Online]. Available: https://www.portescap.com/-/media/project/automation-specialty/portescap/portescap/pdf/specification-pdfs/specifications-35l048b.pdf

[4] Optek Technologies, "OPB931W55 Photologic Slotted Optical Swhich," Datasheet. [Online]. Available: https://www.alldatasheet.com/datasheet-pdf/view/116515/OPTEK/OPB931W55.html. [Accessed: Oct. 8, 2025].

[5] Arduino, "Arduino Nano Technical Details," Hardware Documentation. [Online]. Available: https://docs.arduino.cc/hardware/nano. [Accessed: Oct. 8, 2025].

Appendices

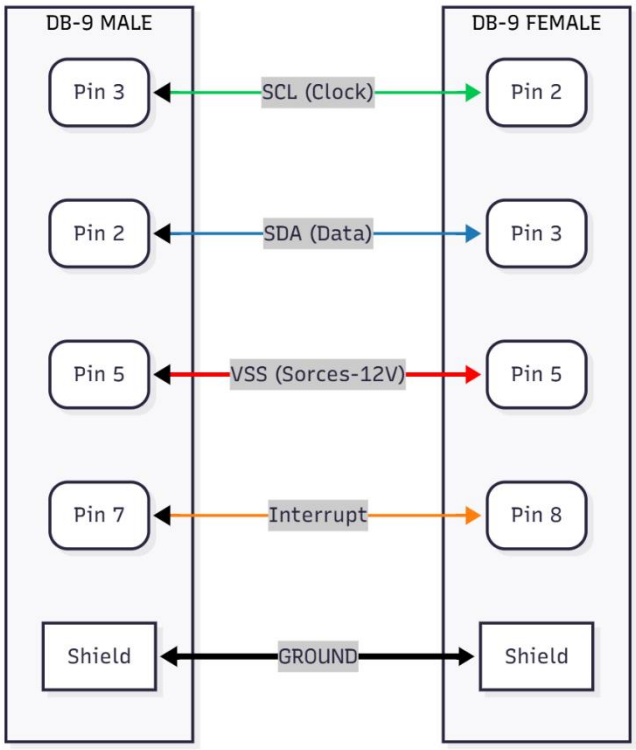

Figure 5: Cable Pin Connections Between Centers

Kaynak (Board/Modül)	Pin	Hedef	Açıklama
Arduino Nano	A4 (SDA)	PCF8574 SDA	I²C data
Arduino Nano	A5 (SCL)	PCF8574 SCL	I²C clock
Arduino Nano	5V	PCF8574 VCC, L293D Vcc1, OPB LED via resistor	Logic supply
Arduino Nano	GND	All module grounds	Common ground required
Arduino Nano	D5 - D8	L293D IN1 - IN4	Stepper control (or via PCF8574 P2,P3...)
L293D	Vcc2	12V Supply	Motor power (use common GND)
L293D	OUT1/OUT2	Stepper Coil A (Yellow/Black)	Check motor color code
L293D	OUT3/OUT4	Stepper Coil B (Red/Gray)	Check motor color code
OPB931	Phototransistor out	Arduino D2 (INT)	Use 10k pull-up to 5V
E119932 cable	—	—	Assume straight-through; verify with continuity test

Figure 6: All Pins Output and Input Schematic

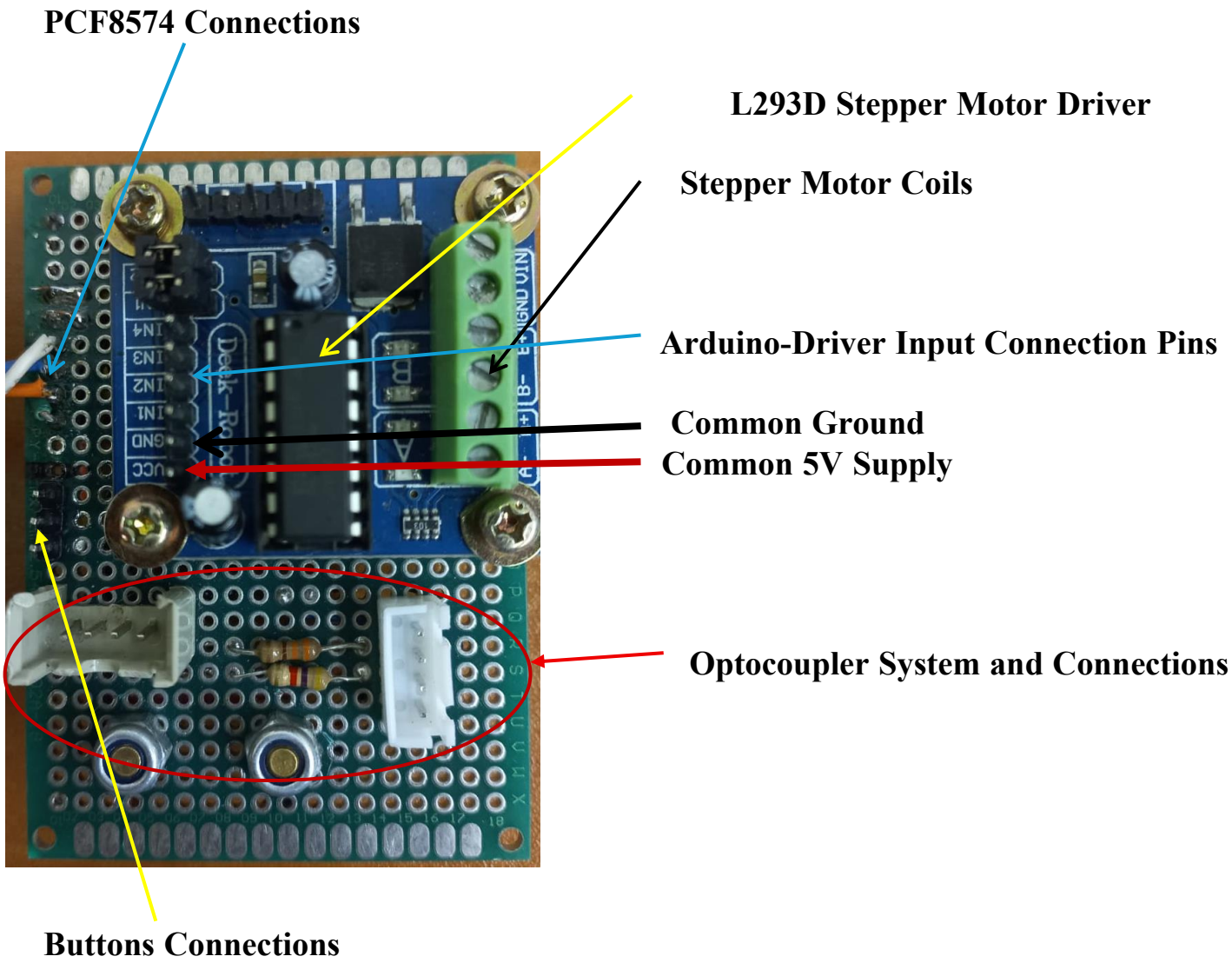

Figure 7: Setup Interior and Components

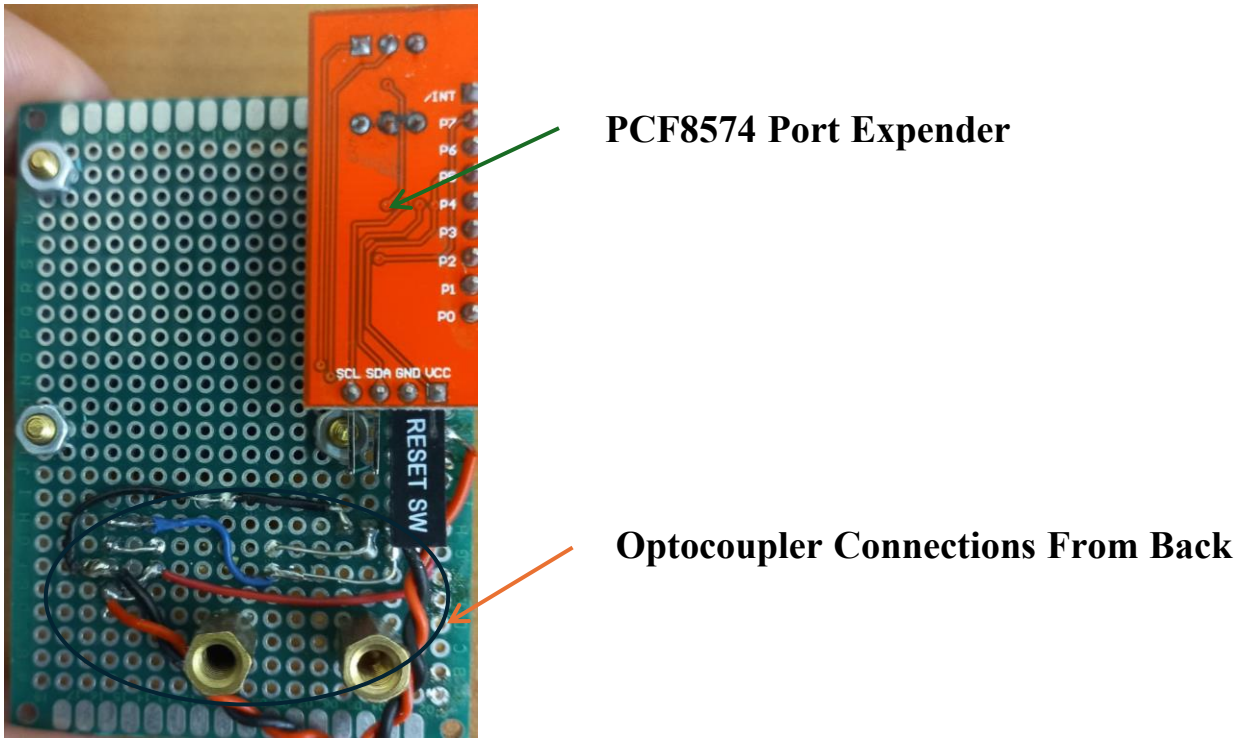

Figure 8: Setup Interior and Components From Back

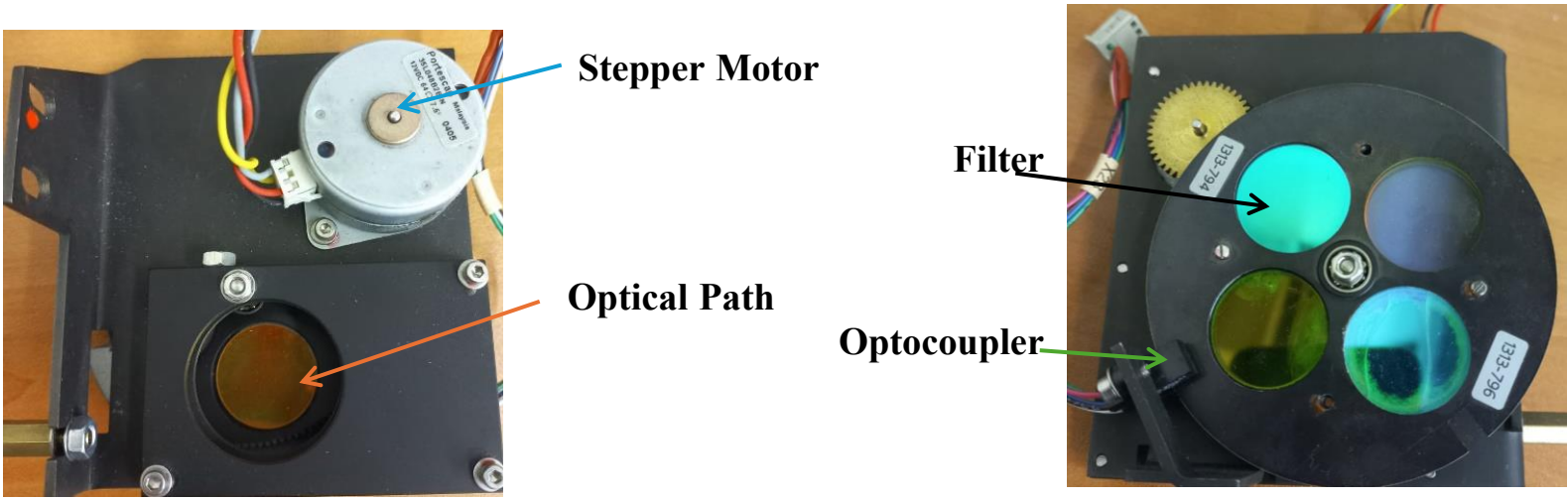

Figure 9-10: Filter Wheel Schematic From Front and Back

Arduino Codes for Project:

```
#include <Wire.h>                // I²C communication library
#include <PCF8574.h>             // Library for the I²C port expander
#include <AccelStepper.h>        // Library for stepper motor control
#include <SerialCommand.h>       // Serial command parser
#include <LiquidCrystal_I2C.h>   // Library for I²C LCD display

// --- Stepper motor step positions for each optical filter slot ---
#define F1 39
#define F2 111
#define F3 183
#define F4 255

// --- Input pins definitions ---
#define sensorPin 3          // Home sensor input (connected to Arduino interrupt pin)
#define buttonFilterPin P4   // Clockwise filter change button (on PCF8574)
#define buttonFilterPin1 P5  // Counter-clockwise filter change button (on PCF8574)
#define buttonHomePin P6     // Home button (on PCF8574)

// --- I²C expander address ---
PCF8574 pcf(0x38);

// --- Stepper motor parameters ---
int valueS = 12000;  // Maximum motor speed in steps/sec
int valueA = 5000;   // Acceleration in steps/sec²
int speed = 350;     // Speed used during homing
int dt = 50;         // Debounce delay (ms)

// --- State variables ---
int pressCount = 0;  // Keeps track of selected filter index
bool lastFilterButton = LOW, lastFilterButton1 = LOW, lastHomeButton = LOW;
volatile bool homeFound = false; // Flag set when the home sensor triggers

// --- Mechanical configuration ---
const int stepsPerRevolution = 96; // Motor steps per one full revolution
const int totalGearSteps = 288;    // Total steps for one full rotation of the filter wheel

// --- Hardware objects ---
LiquidCrystal_I2C lcd(0x27, 16, 2); // I²C LCD module (address 0x27, 16x2)
SerialCommand sCmd;                 // Serial command handler

// --- Custom stepper class using PCF8574 I/O expander ---
class PCFStepper : public AccelStepper {
public:
  PCFStepper() : AccelStepper(AccelStepper::HALF4WIRE, 0, 0, 0, 0, false) {}

protected:
  void setOutputPins(uint8_t mask) override {
    // Override to control stepper coil pins via PCF8574 instead of Arduino GPIO
    pcf.digitalWrite(0, (mask & 0b0001) ? HIGH : LOW);
    pcf.digitalWrite(1, (mask & 0b0010) ? HIGH : LOW);
    pcf.digitalWrite(2, (mask & 0b0100) ? HIGH : LOW);
    pcf.digitalWrite(3, (mask & 0b1000) ? HIGH : LOW);
  }
};
```

```
// --- Stepper object (driven through PCF8574) ---
PCFStepper stepper;

void setup() {
  Wire.begin();
  pcf.begin();
  Serial.begin(9600);
  lcd.begin();
  lcd.backlight();

  // Configure input pin (home sensor) with pull-up resistor
  pinMode(sensorPin, INPUT_PULLUP);

  // Initialize PCF8574 pins
  // Pins 0-3 control the stepper motor coils (outputs)
  for (int i = 0; i < 4; i++) {
    pcf.pinMode(i, OUTPUT);
    pcf.digitalWrite(i, LOW); // Default LOW state
  }

  // Buttons are configured as inputs
  pcf.pinMode(buttonFilterPin, INPUT);
  pcf.pinMode(buttonFilterPin1, INPUT);
  pcf.pinMode(buttonHomePin, INPUT);

  lastHomeButton = pcf.digitalRead(buttonHomePin);

  // Display available serial commands
  Serial.println("Commands: F1, F2, F3, F4, Home, SPEED, ACCEL, Where");

  // Map serial commands to their corresponding functions
  sCmd.addCommand("F1", selectFirst);
  sCmd.addCommand("F2", selectSecond);
  sCmd.addCommand("F3", selectThird);
  sCmd.addCommand("F4", selectFourth);
  sCmd.addCommand("Home", goHome);
  sCmd.addCommand("SPEED", cmdSetS);
  sCmd.addCommand("ACCEL", cmdSetA);
  sCmd.addCommand("Where", where);
  sCmd.setDefaultHandler(unrecognized);

  // Set initial maximum speed
  stepper.setMaxSpeed(valueS);

  // Automatically find the home position on startup
  findHomePosition();
}

// --- Displays the current filter position ---
void where() {
  int pos = stepper.currentPosition();
  long Fpos = pos % totalGearSteps;

  if (Fpos == F1) { lcd.clear(); lcd.print("On filter 1"); Serial.println("On filter 1");
}
  if (Fpos == F2) { lcd.clear(); lcd.print("On filter 2"); Serial.println("On filter 2");
}
  if (Fpos == F3) { lcd.clear(); lcd.print("On filter 3"); Serial.println("On filter 3");
}
```

```
  if (Fpos == F4 || Fpos == -33) { lcd.clear(); lcd.print("On filter 4"); Serial.println("On
filter 4"); }
  if (Fpos == 0) { lcd.clear(); lcd.print("On Home"); Serial.println("On Home"); }
}

// --- Interrupt Service Routine: triggered by the home sensor ---
void homeInterrupt() {
  homeFound = true;
}

// --- Function to locate the home position automatically ---
void findHomePosition() {
  attachInterrupt(digitalPinToInterrupt(sensorPin), homeInterrupt, RISING);
  stepper.setSpeed(speed);

  while (!homeFound) {
    stepper.runSpeed();
  }
  stepper.stop();

  if (homeFound) {
    Serial.println("Home found. Ready for filters.");
    stepper.setCurrentPosition(0);
    lcd.clear();
    lcd.print("On Home");
    lcd.setCursor(0,1);
    lcd.print("READY");
  } else {
    Serial.println("Home not found.");
  }
  homeFound = false;
}

// --- Serial command: set motor speed ---
void cmdSetS() {
  char *arg = sCmd.next();
  if(arg != NULL){
    valueS = atoi(arg);
    stepper.setMaxSpeed(valueS);
    Serial.print("Speed set to: "); Serial.println(valueS);
  } else Serial.println("No parameter!");
}

// --- Serial command: set motor acceleration ---
void cmdSetA() {
  char *arg = sCmd.next();
  if(arg != NULL){
    valueA = atoi(arg);
    stepper.setAcceleration(valueA);
    Serial.print("Acceleration set to: "); Serial.println(valueA);
  } else Serial.println("No parameter!");
}

// --- Moves the stepper to the target filter position ---
void moveToFilter(int targetStep, const char* label) {
  stepper.setMaxSpeed(valueS);
  stepper.setAcceleration(valueA);

  long currentPos = stepper.currentPosition();
  long diff = targetStep - (currentPos % totalGearSteps);
```

```
  // Handle shortest path rotation
  if(diff > totalGearSteps/2) diff -= totalGearSteps;
  if(diff < -totalGearSteps/2) diff += totalGearSteps;
  if(diff == 216) diff = -72;

  stepper.move(diff);
  stepper.runToPosition();

  lcd.clear();
  lcd.print("On "); lcd.print(label);
  Serial.print("On "); Serial.println(label);
}

// --- Filter selection shortcuts ---
void selectFirst() { moveToFilter(F1, "F1"); }
void selectSecond() { moveToFilter(F2, "F2"); }
void selectThird() { moveToFilter(F3, "F3"); }
void selectFourth() { moveToFilter(F4, "F4"); }

// --- Returns to home position ---
void goHome() { findHomePosition(); }

// --- Handles unrecognized serial commands ---
void unrecognized(const char *command) {
  Serial.print("Unknown command: "); Serial.println(command);
  Serial.println("Use: F1, F2, F3, F4, Home, SPEED, ACCEL, Where");
}

// --- Moves circularly between filters using button presses ---
void goToFilterCircular(int filterNumber) {
  int targetStep = 0;
  const char* label = "";

  if(filterNumber == 1){targetStep = F1; label = "F1";}
  if(filterNumber == 2){targetStep = F2; label = "F2";}
  if(filterNumber == 3){targetStep = F3; label = "F3";}
  if(filterNumber == 4){targetStep = F4; label = "F4";}

  moveToFilter(targetStep, label);
}

// --- Main loop: handles buttons and serial commands ---
void loop() {
  sCmd.readSerial(); // Continuously listen for serial commands

  // Read the state of PCF8574 buttons
  bool currentFilterButton = (pcf.digitalRead(buttonFilterPin)==HIGH);
  bool currentFilterButton1 = (pcf.digitalRead(buttonFilterPin1)==HIGH);
  bool currentHomeButton = (pcf.digitalRead(buttonHomePin)==HIGH);

  // Clockwise filter rotation
  if(currentFilterButton==HIGH && lastFilterButton==LOW){
    pressCount++;
    if(pressCount>4) pressCount=1;
    goToFilterCircular(pressCount);
  }

  // Counter-clockwise rotation
  if(currentFilterButton1==HIGH && lastFilterButton1==LOW){
```

```
    pressCount--;
    if(pressCount<1) pressCount=4;
    goToFilterCircular(pressCount);
  }

  // Home button rising edge
  if(currentHomeButton==HIGH && lastHomeButton==LOW){
    findHomePosition();
  }

  // Update button state memory
  lastHomeButton=currentHomeButton;
  lastFilterButton=currentFilterButton;
  lastFilterButton1=currentFilterButton1;

  delay(dt); // Small delay for button debounce
}
```